

Artificial Intelligence

Instructor: Kietikul Jearanaitanakij

Department of Computer Engineering

King Mongkut's Institute of Technology Ladkrabang

Lecture 10

Genetic Algorithm

- Evolution
- Steps of Genetic Algorithm
- Roulette wheel selection
- Crossover operator
- Mutation operator
- GA case studies

Genetic Algorithm (GA)

- In the early 1970s, John Holland introduced the concept of genetic algorithms.
- GA combines three concepts of **evolution**, **natural selection**, and **genetics** to search for the optimal solution.
- Optimization **iteratively improves the quality of solutions** until the optimal solution is found.
- We use the **fitness function** to measure the quality of a solution.

- GA represents a solution as a **chromosome** consisting of some “**genes**”.
- Each **gene** is represented by 0 or 1:

1	0	1	1	0	1	0	0	0	0	0	1	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- **Steps in GA**

1. Create a **population of chromosomes**.
2. **Evaluate their fitness**.
3. Create a new population through **genetic operations**.
4. **Repeat this process** many times.

- **Genetic operations**

1. **Crossover** exchanges parts of two single chromosomes.
2. **Cloning** copies chromosomes.
3. **Mutation** changes the gene value in some random locations of a chromosome.

Full Steps of Genetic Algorithm

- **Step 1:**
 - Encode a solution as a **chromosome** of a fixed length.
 - Define **population size N** , **crossover probability p_c** , and **mutation probability p_m** .
- **Step 2:** Define a **fitness function** to measure the performance, or fitness, of an individual chromosome in the problem domain.
- **Step 3:** Randomly generate an **initial population** of chromosomes of size N :
 $\{ x_1, x_2, \dots, x_N \}$

- **Step 4**: Calculate the fitness of each chromosome:

$$f(x_1), f(x_2), \dots, f(x_N)$$

- **Step 5**: Select a pair of chromosomes for mating from the current population.
- **Step 6**: Create a pair of offspring chromosomes by applying the genetic operators – **crossover/cloning** and **mutation**.
- **Step 7**: Place the created offspring chromosomes in the **new population**.
- **Step 8**: Repeat Step 5 until the size of the new chromosome population becomes equal to the size of the initial population, **N**.

- **Step 9**: Replace the initial (parent) chromosome population with the new (offspring) population.
 - **Step 10**: Go to Step 4, and repeat the process until the termination criterion is satisfied.
-
- **Each iteration** is called a **generation**. A typical number of generations for a simple GA can range from 50 to over 500.
 - The **entire set of generations** is called a **run**.
 - GA returns the best chromosomes in the population as a final solution.

The genetic algorithm cycle

Initial population

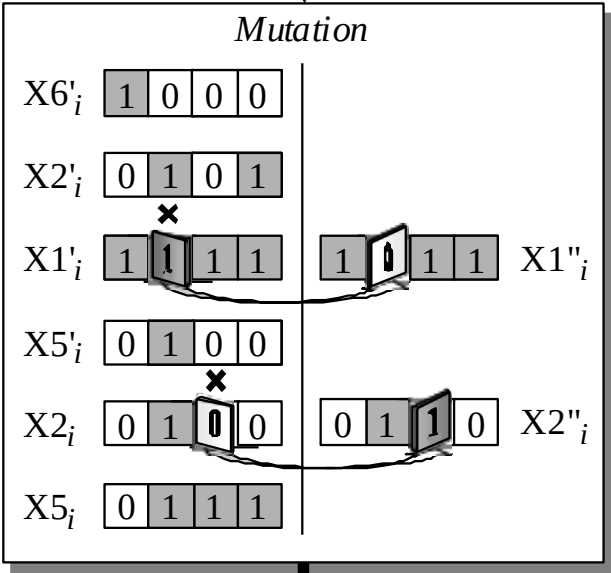
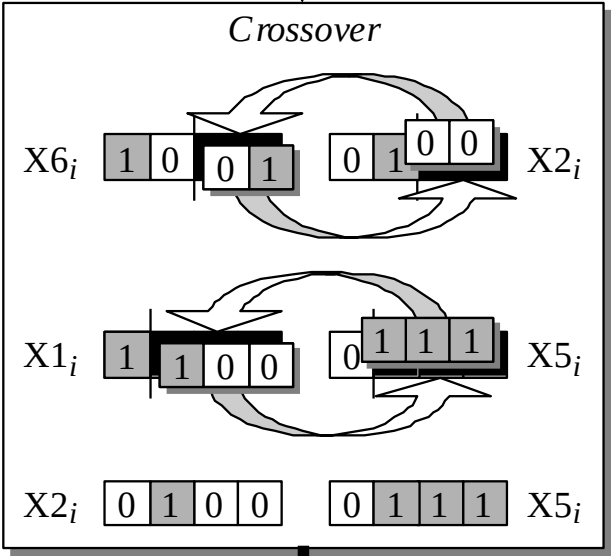
Generation i

$X1_i$	1	1	0	0	$f = 36$
$X2_i$	0	1	0	0	$f = 44$
$X3_i$	0	0	0	1	$f = 14$
$X4_i$	1	1	1	0	$f = 14$
$X5_i$	0	1	1	1	$f = 56$
$X6_i$	1	0	0	1	$f = 54$

New population

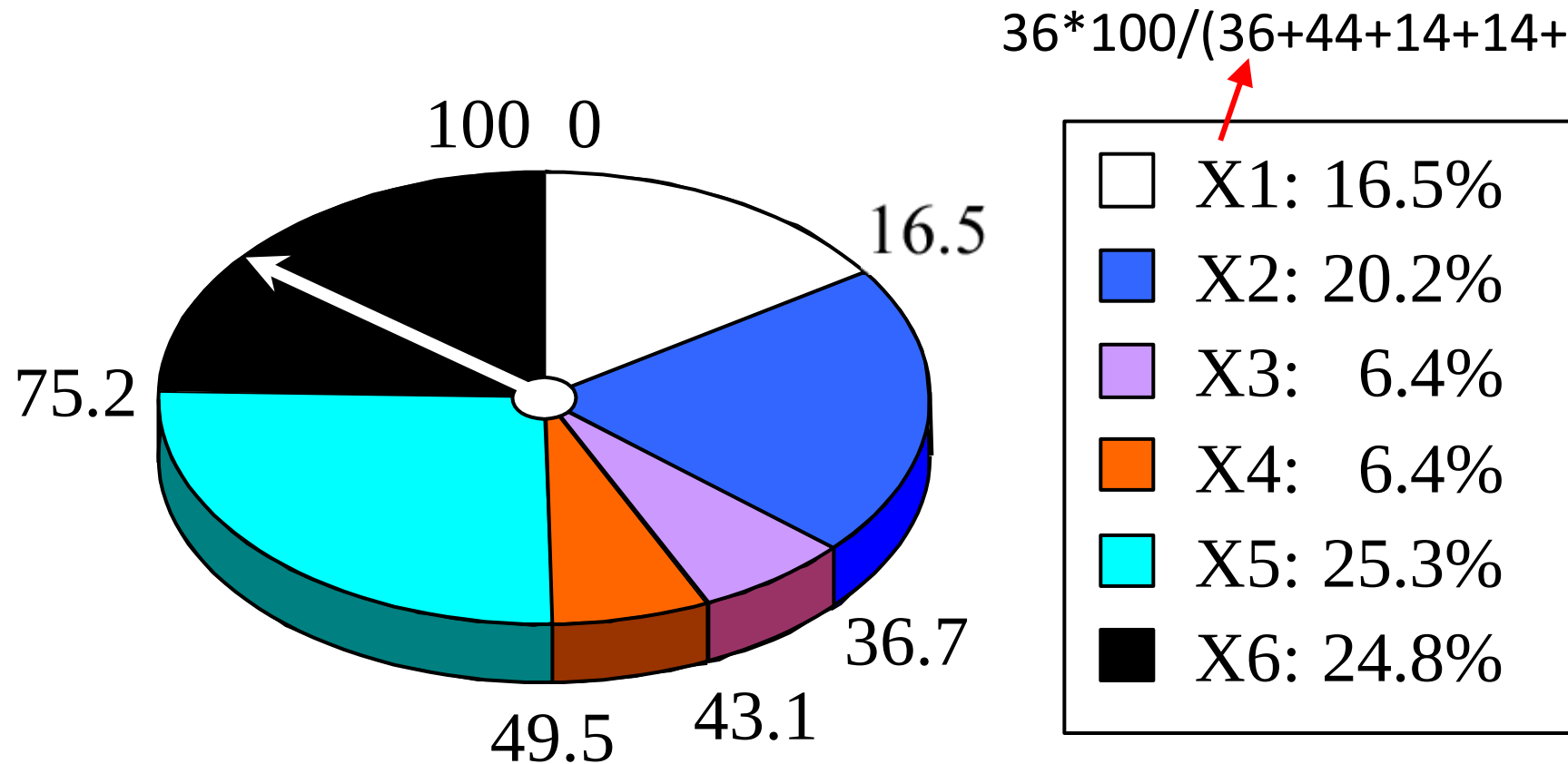
Generation $(i + 1)$

$X1_{i+1}$	1	0	0	0	$f = 56$
$X2_{i+1}$	0	1	0	1	$f = 50$
$X3_{i+1}$	1	0	1	1	$f = 44$
$X4_{i+1}$	0	1	0	0	$f = 44$
$X5_{i+1}$	0	1	1	0	$f = 54$
$X6_{i+1}$	0	1	1	1	$f = 56$



How to select chromosomes?

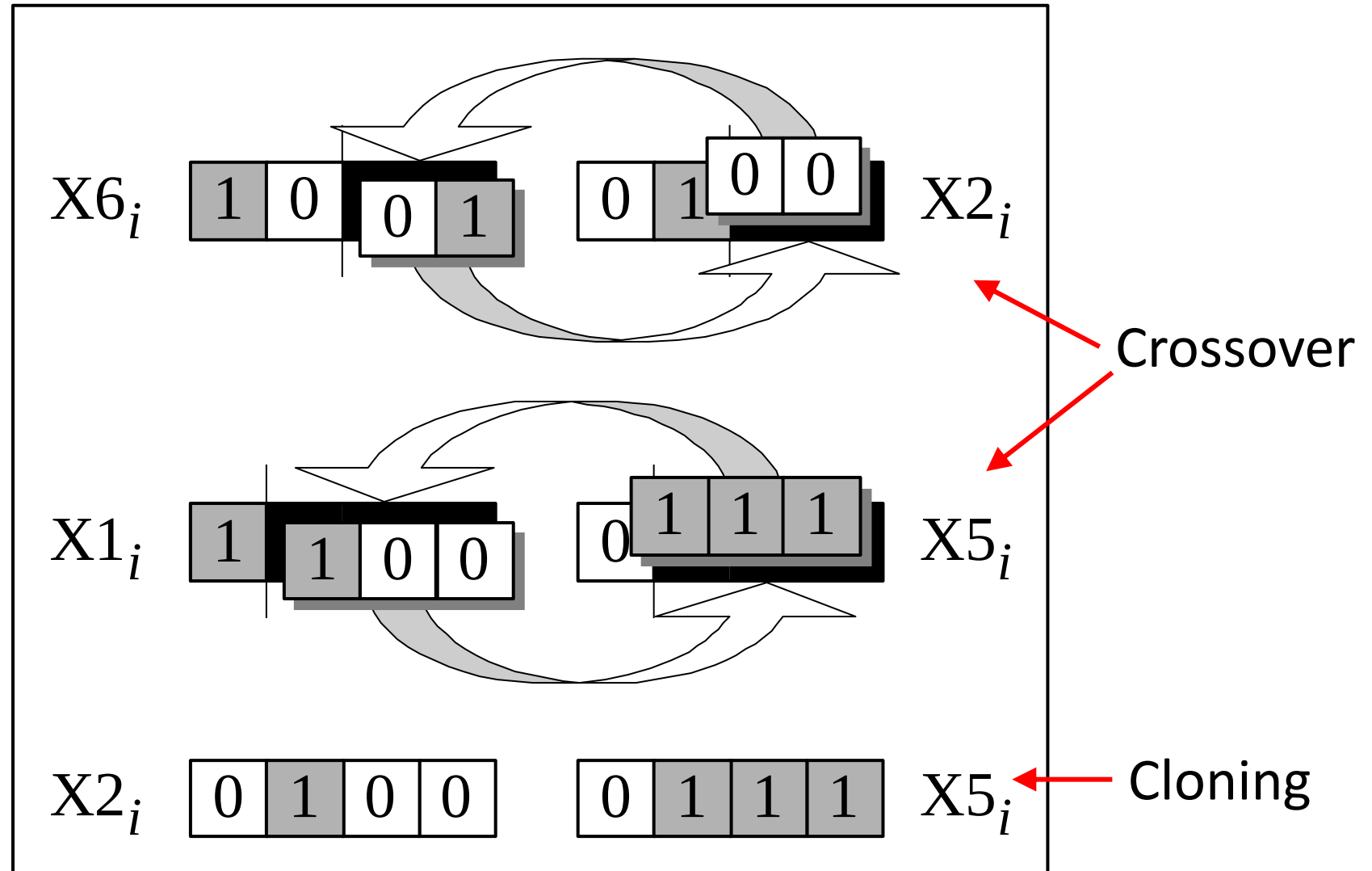
The most commonly used chromosome selection technique is the **roulette wheel selection**.



Crossover/Cloning operator

- **IF** $\text{random} \leq \text{crossover probability } p_c$, then do the crossover.
 - The **crossover** operator randomly chooses a crossover point where two parent chromosomes “break”.
 - Exchanges the chromosome parts.
 - As a result, two new offspring are created.
- **Else** do the chromosome **cloning**. The offspring are created as exact copies of their parents.

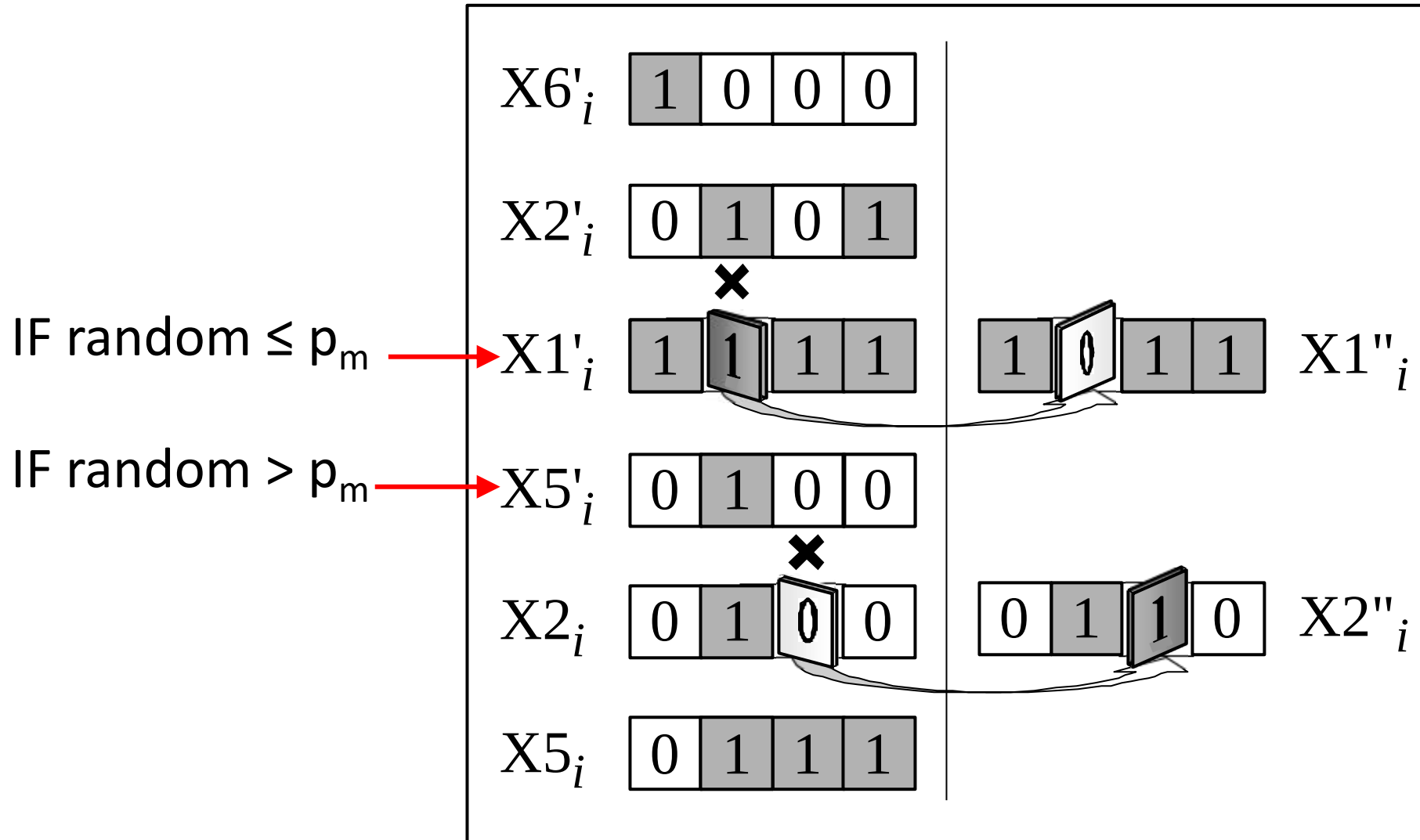
Crossover / Cloning



Mutation operator

- Mutation represents a change in the gene.
- The mutation operator randomly flips a gene in a chromosome.
- Mutation can avoid a trap on a local optima.
- The mutation probability is quite small in nature and is kept low for GAs, typically in the range between 0.001 and 0.01.

Mutation



Genetic algorithms: case study

- Find the maximum value of the function $(15x - x^2)$ where x varies between 0 and 15.
- For simplicity, we assume that x takes only integer values and a chromosome is represented as four genes:

<i>Integer</i>	<i>Binary code</i>	<i>Integer</i>	<i>Binary code</i>	<i>Integer</i>	<i>Binary code</i>
1	0 0 0 1	6	0 1 1 0	11	1 0 1 1
2	0 0 1 0	7	0 1 1 1	12	1 1 0 0
3	0 0 1 1	8	1 0 0 0	13	1 1 0 1
4	0 1 0 0	9	1 0 0 1	14	1 1 1 0
5	0 1 0 1	10	1 0 1 0	15	1 1 1 1

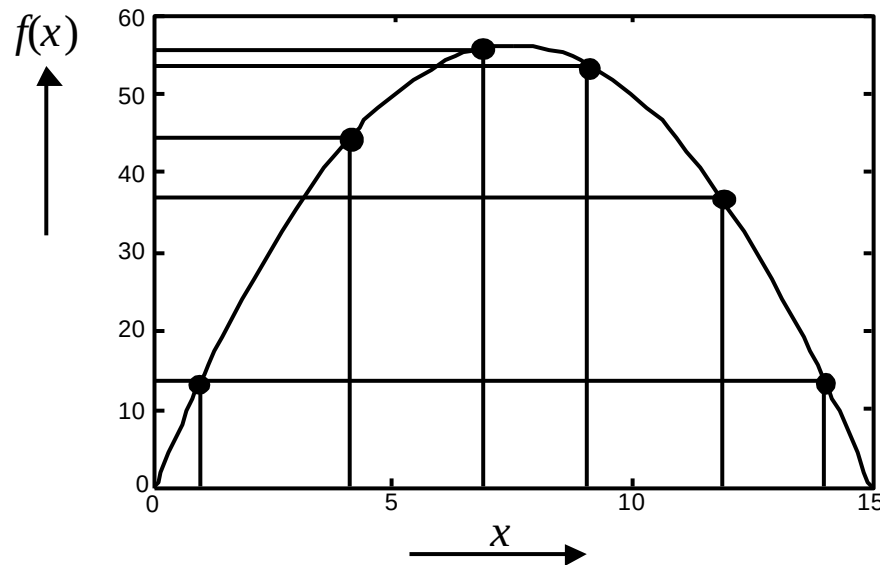
Suppose that :

- The size of the chromosome population N is 6.
- The crossover probability p_c equals 0.7.
- The mutation probability p_m equals 0.001.
- The fitness function in our example is defined by

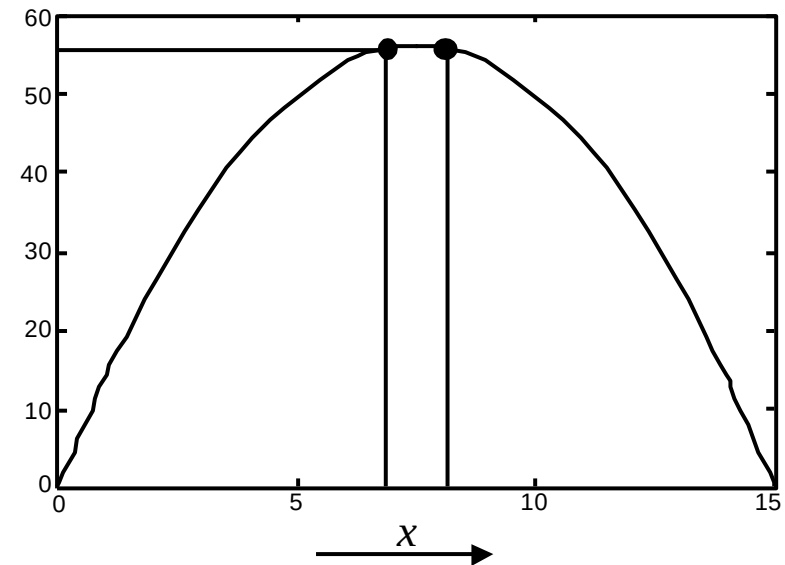
$$f(x) = 15x - x^2$$

Initial Population: N=6

<i>Chromosome label</i>	<i>Chromosome string</i>	<i>Decoded integer</i>	<i>Chromosome fitness</i>	<i>Fitness ratio, %</i>
X1	1 1 0 0	12	36	16.5
X2	0 1 0 0	4	44	20.2
X3	0 0 0 1	1	14	6.4
X4	1 1 1 0	14	14	6.4
X5	0 1 1 1	7	56	25.7
X6	1 0 0 1	9	54	24.8



(a) Chromosome initial locations.



(b) Chromosome final locations.

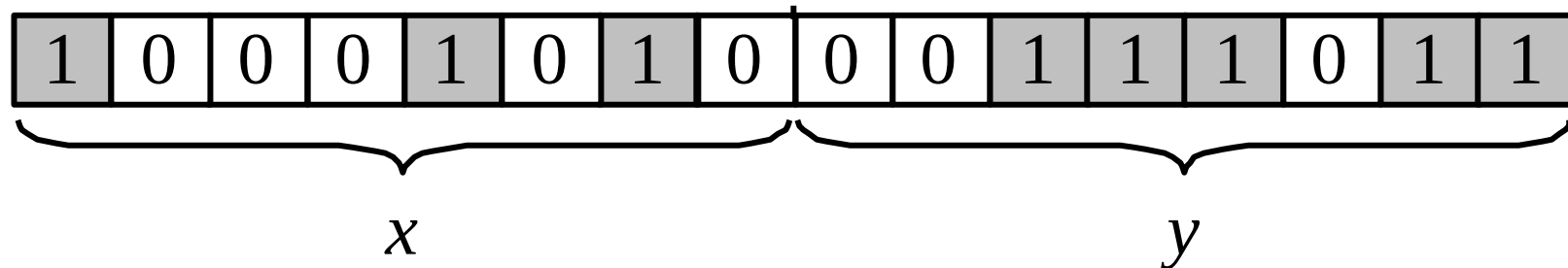
Genetic algorithms: another case study

- Find the maximum of the function of two variables:

$$f(x, y) = (1-x)^2 e^{-x^2-(y+1)^2} - (x-x^3-y^3) e^{-x^2-y^2}$$

where parameters x and y vary between -3 and 3.

- The first step is to represent the problem variables as a chromosome - parameters x and y as a concatenated binary string:



- We also choose the **size of the chromosome population** and randomly generate an initial population.
- The next step is to **calculate the fitness** of each chromosome. This is **done in two stages**.

- First, extract 2 strings, X and Y, from a chromosome:



- Then convert both X and Y from a binary number to a decimal number:

$$(10001010)_2 = 1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = (138)_{10}$$

and

$$(00111011)_2 = 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = (59)_{10}$$

- Now the range of integers that can be handled by 8-bits, that is the range from 0 to $(2^8 - 1)$, is mapped to the actual range of parameters x and y , that is the range from -3 to 3:

Real range $\leftarrow \frac{6}{256-1} = 0.0235294$

Chromosome range $\leftarrow$

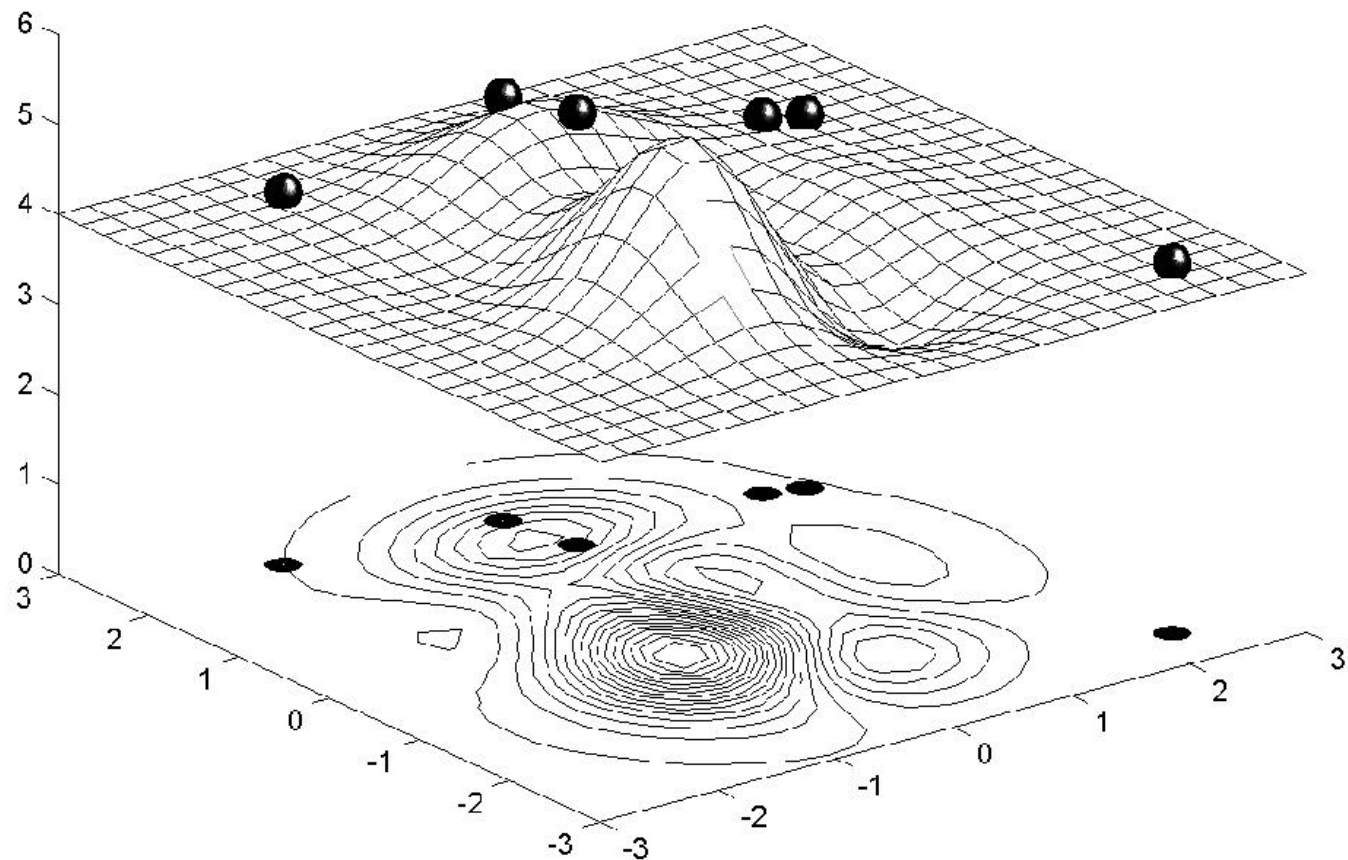
- To obtain the actual values of x and y , we multiply their decimal values by 0.0235294 and subtract 3 from the results:

$$x = (138)_{10} \times 0.0235294 - 3 = 0.2470588$$

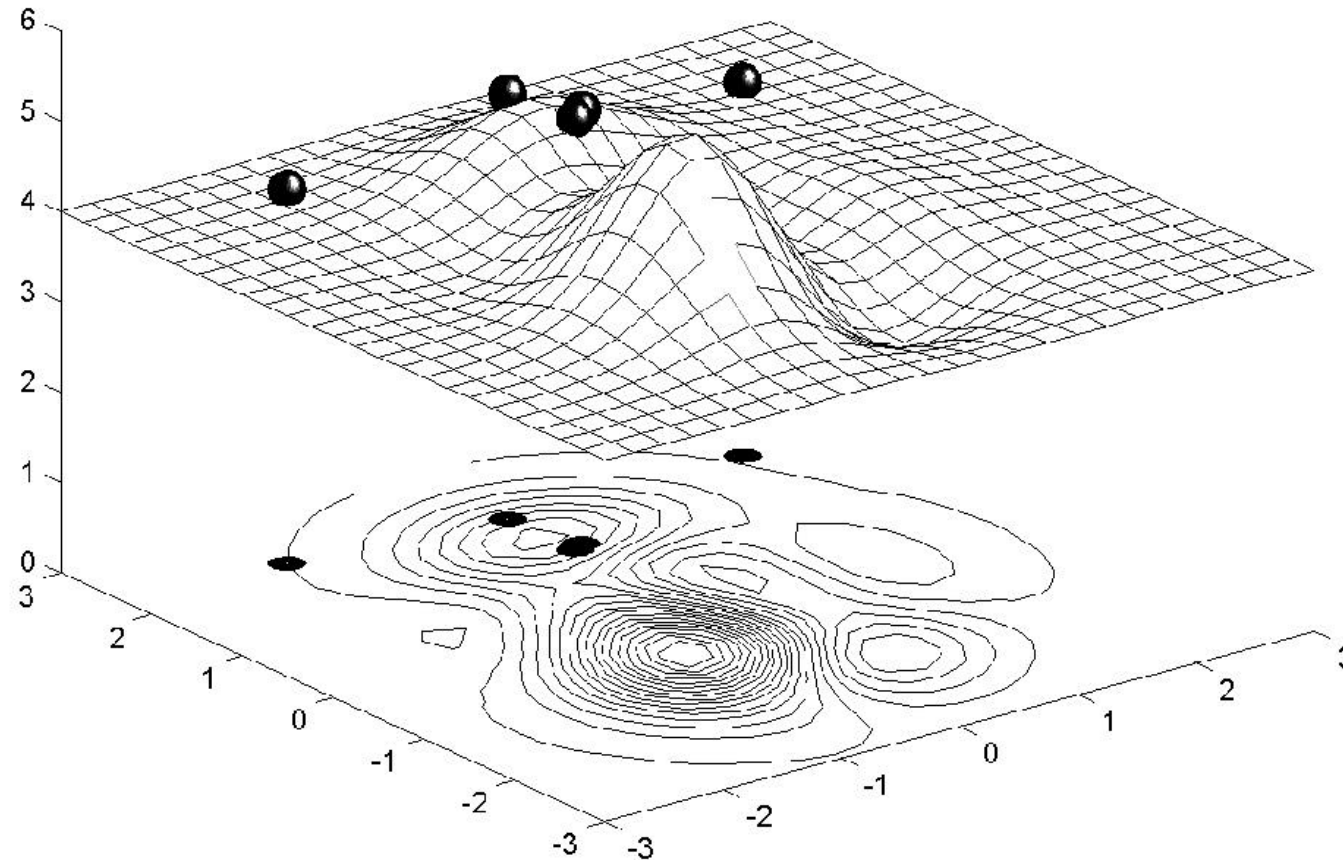
and

$$y = (59)_{10} \times 0.0235294 - 3 = -1.6117647$$

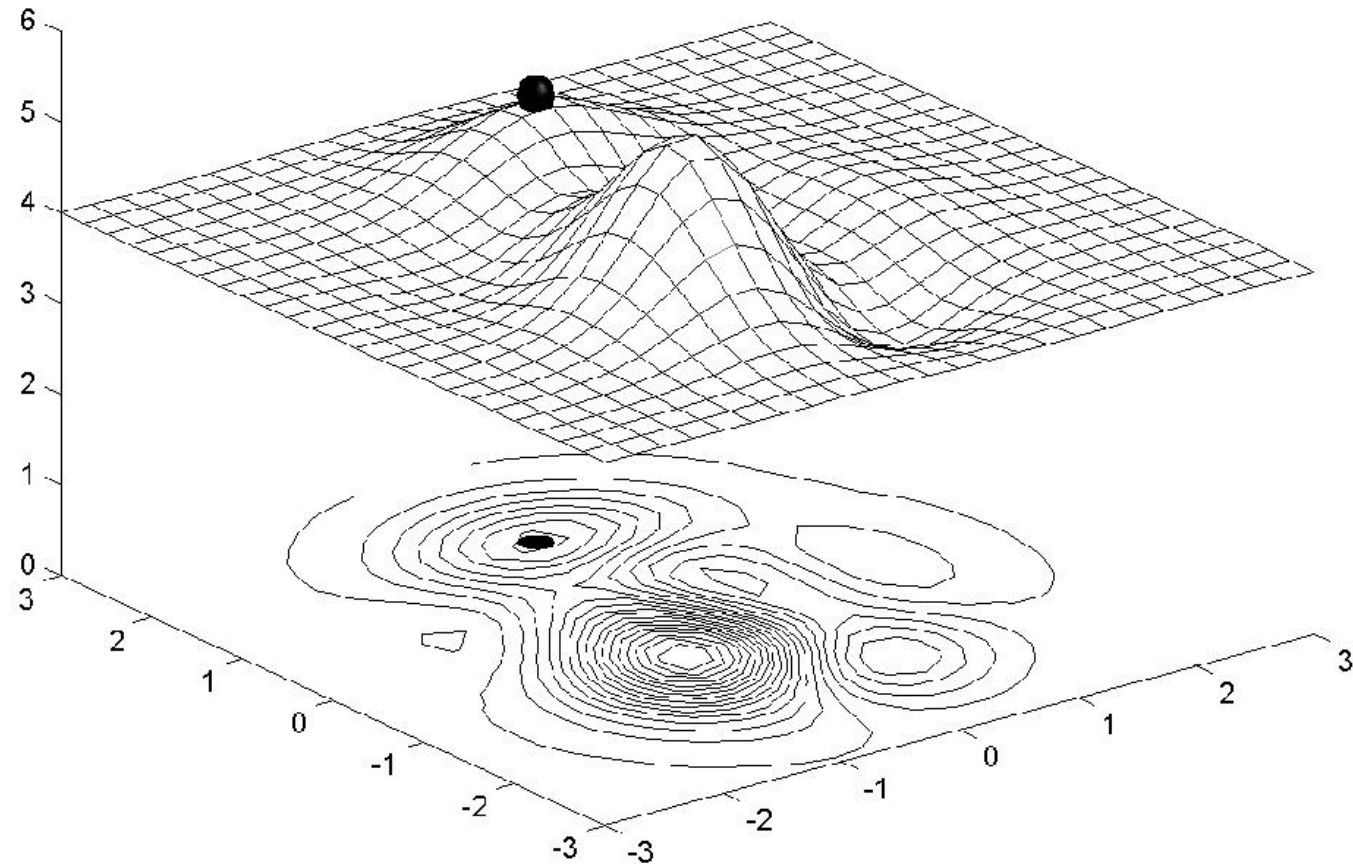
Chromosome locations on the surface of the “peak” function: initial population



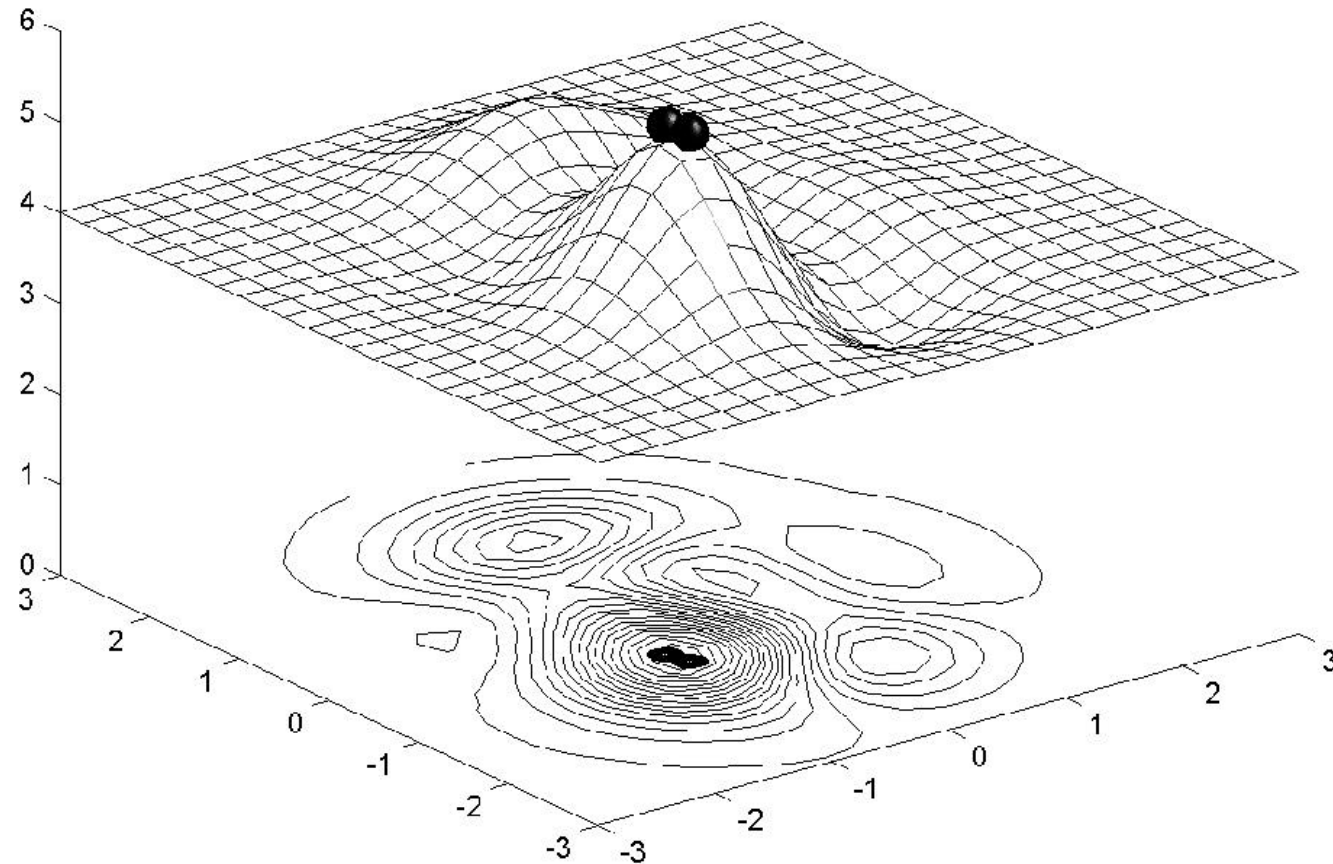
Chromosome locations on the surface of the “peak” function: initial population



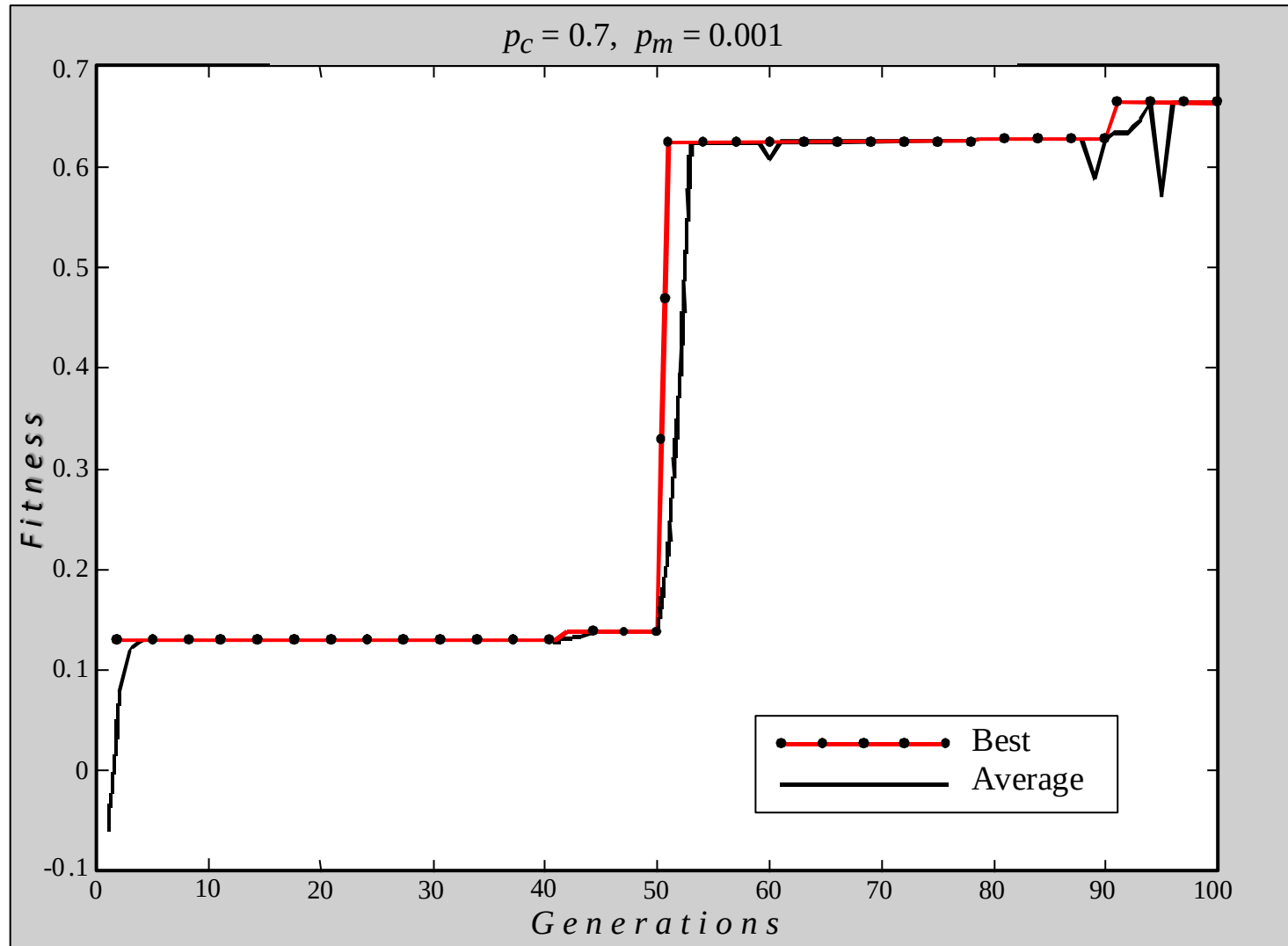
Chromosome locations on the surface of the “peak” function: initial population



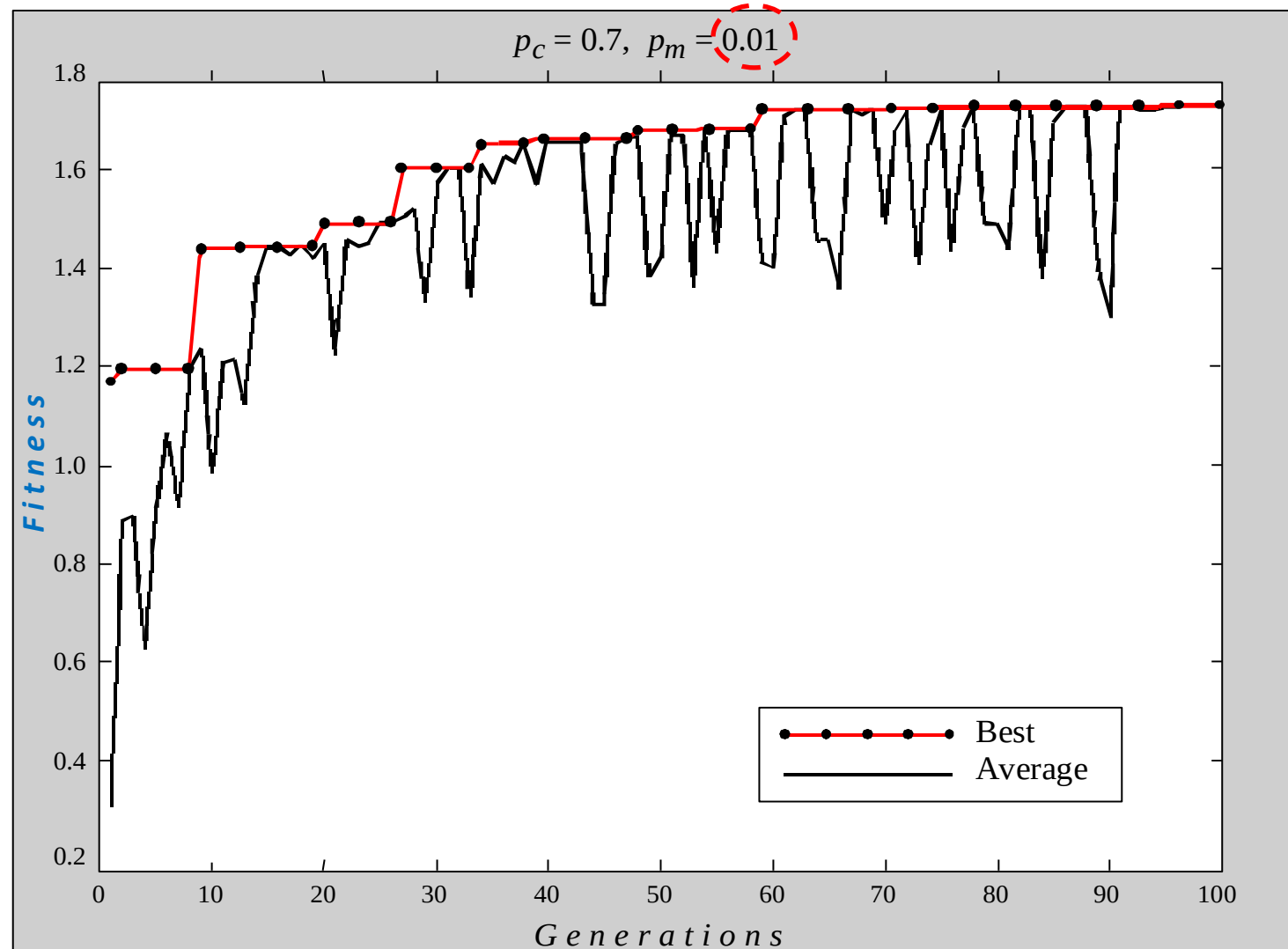
Chromosome locations on the surface of the “peak” function: initial population



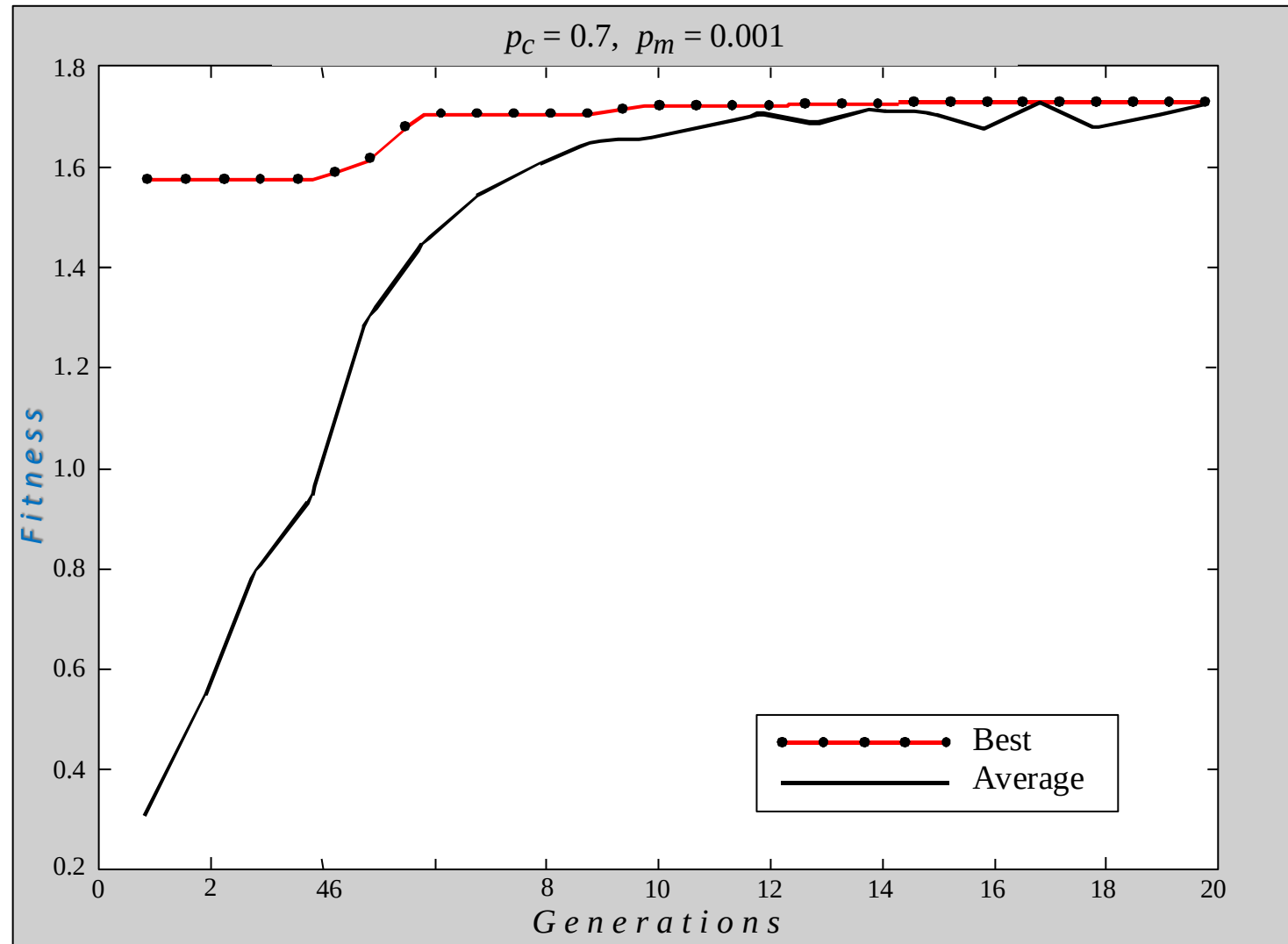
Performance graphs for 100 generations of 6 chromosomes: local maximum



Performance graphs for 100 generations of 6 chromosomes: **global maximum**



Performance graphs for 20 generations of 60 chromosomes



Case study: Scheduling of 7 power units in 4 quarters

The problem constraints:

- The **maximum loads** expected during four quarters are 80, 90, 65, and 70 MW;
- Maintenance of any unit starts at the beginning of an interval and finishes at the end of the same or adjacent interval.
- The **net reserve** of the power system must be **greater or equal to zero at any interval**.

The optimum criterion is the **maximum of the net reserve at any maintenance period**.

Power units and maintenance requirements

Unit number	Unit capacity , MW	Number of quarters required for unit maintenance
1	20	2
2	15	2
3	35	1
4	40	1
5	15	1
6	15	1
7	10	1

Total capacity : $20+15+35+40+15+15+10 = 150$ MW

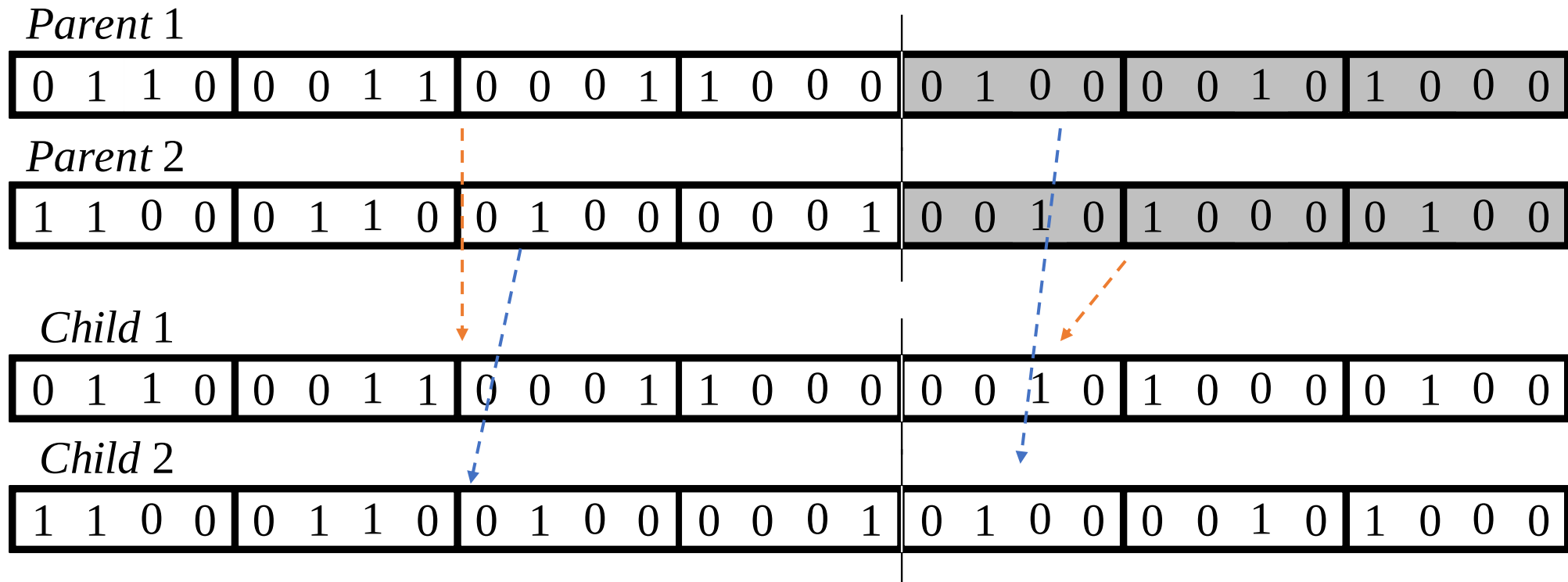
Legal unit gene pools

Unit 1:	1 1 0 0	0 1 1 0	0 0 1 1	
Unit 2:	1 1 0 0	0 1 1 0	0 0 1 1	
Unit 3:	1 0 0 0	0 1 0 0	0 0 1 0	0 0 0 1
Unit 4:	1 0 0 0	0 1 0 0	0 0 1 0	0 0 0 1
Unit 5:	1 0 0 0	0 1 0 0	0 0 1 0	0 0 0 1
Unit 6:	1 0 0 0	0 1 0 0	0 0 1 0	0 0 0 1
Unit 7:	1 0 0 0	0 1 0 0	0 0 1 0	0 0 0 1

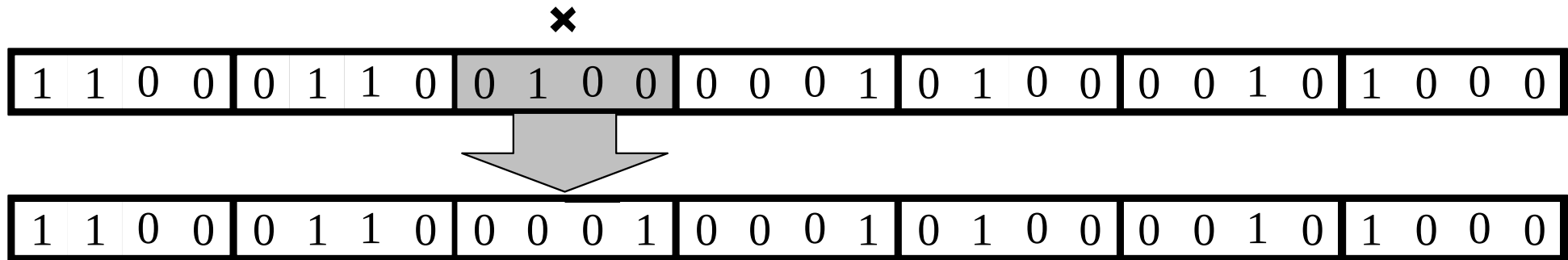
Chromosome for the scheduling problem

Unit 1	Unit 2	Unit 3	Unit 4	Unit 5	Unit 6	Unit 7
0 1 1 0	0 0 1 1	0 0 0 1	1 0 0 0	0 1 0 0	0 0 1 0	1 0 0 0

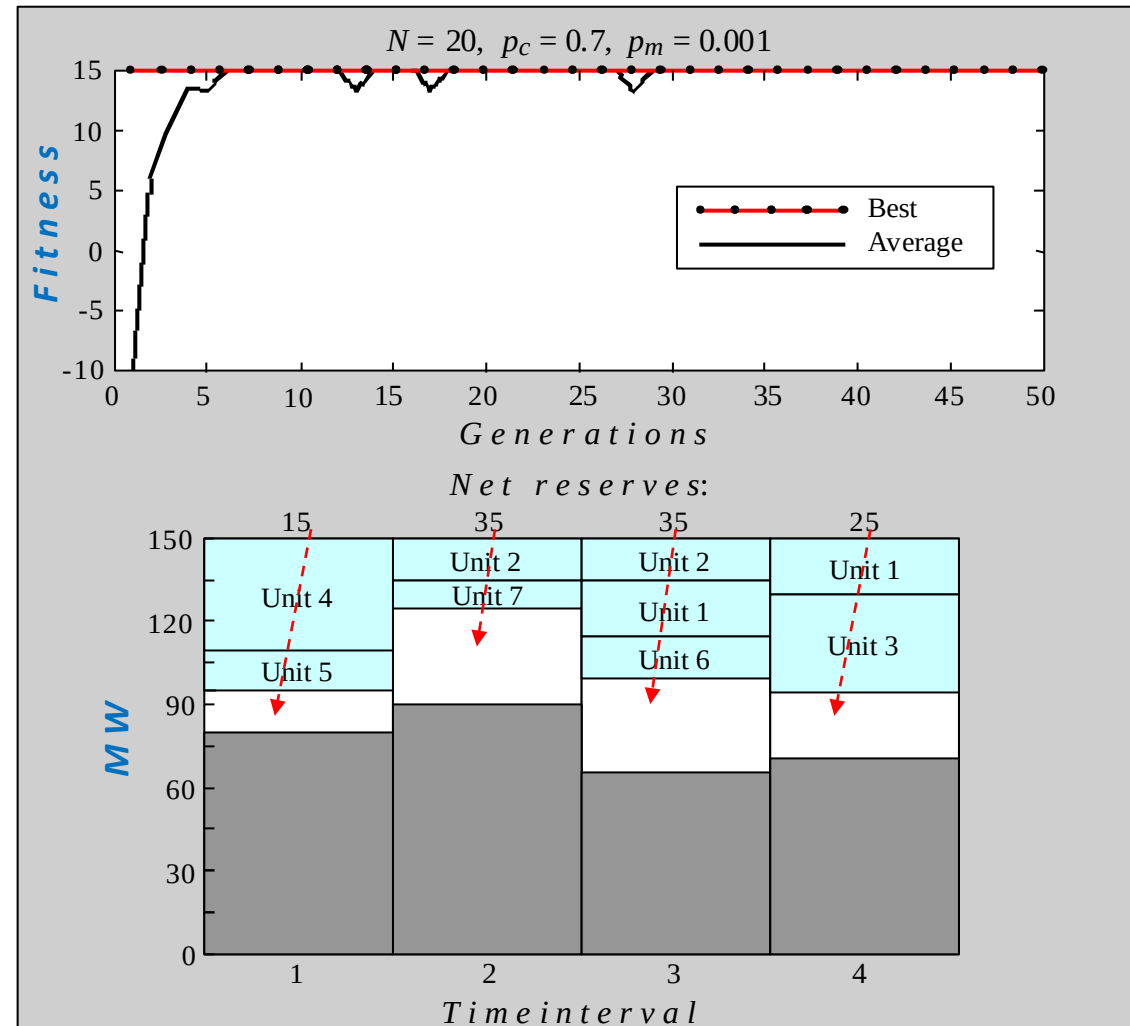
The crossover operator



The mutation operator

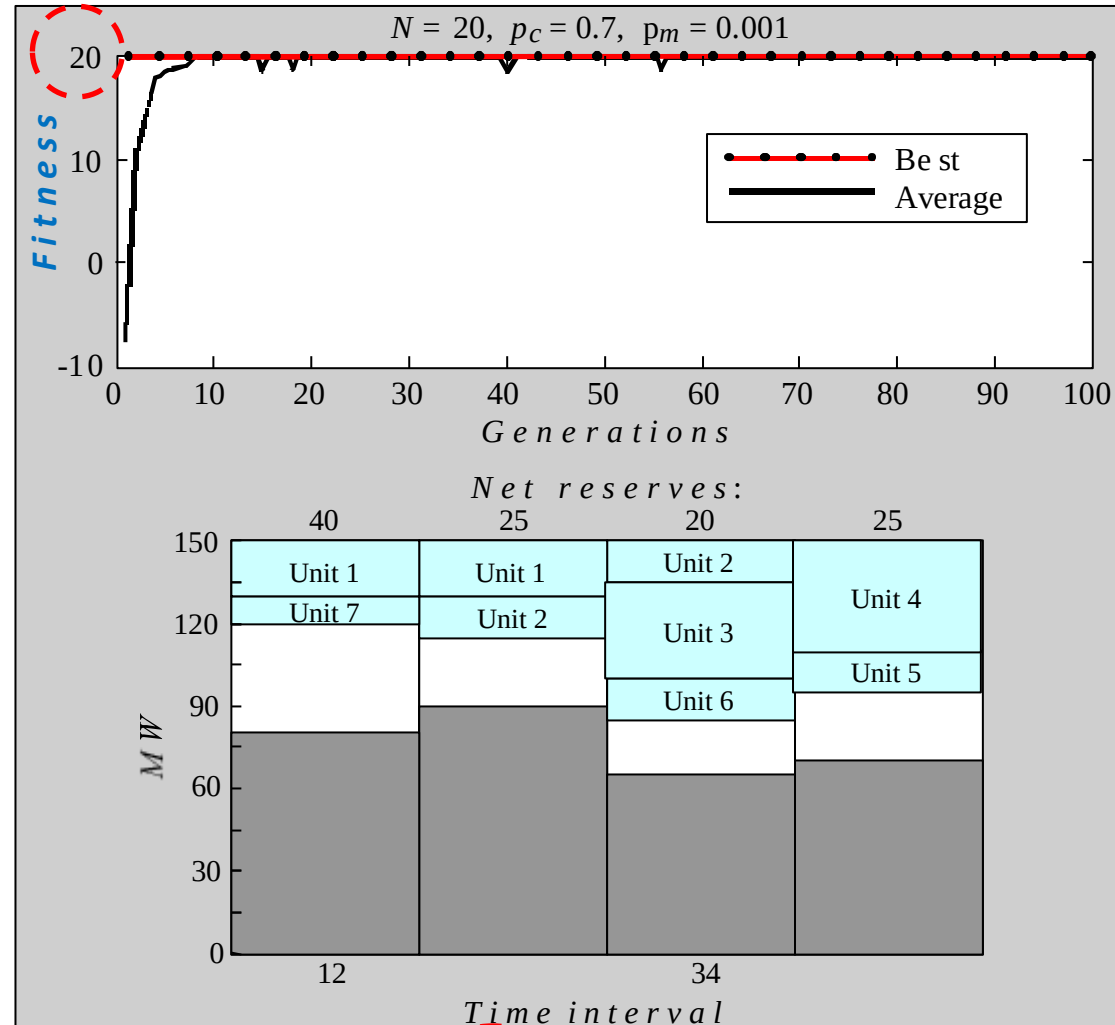


Performance graphs and the best maintenance schedules created in a population of $N=20$ chromosomes



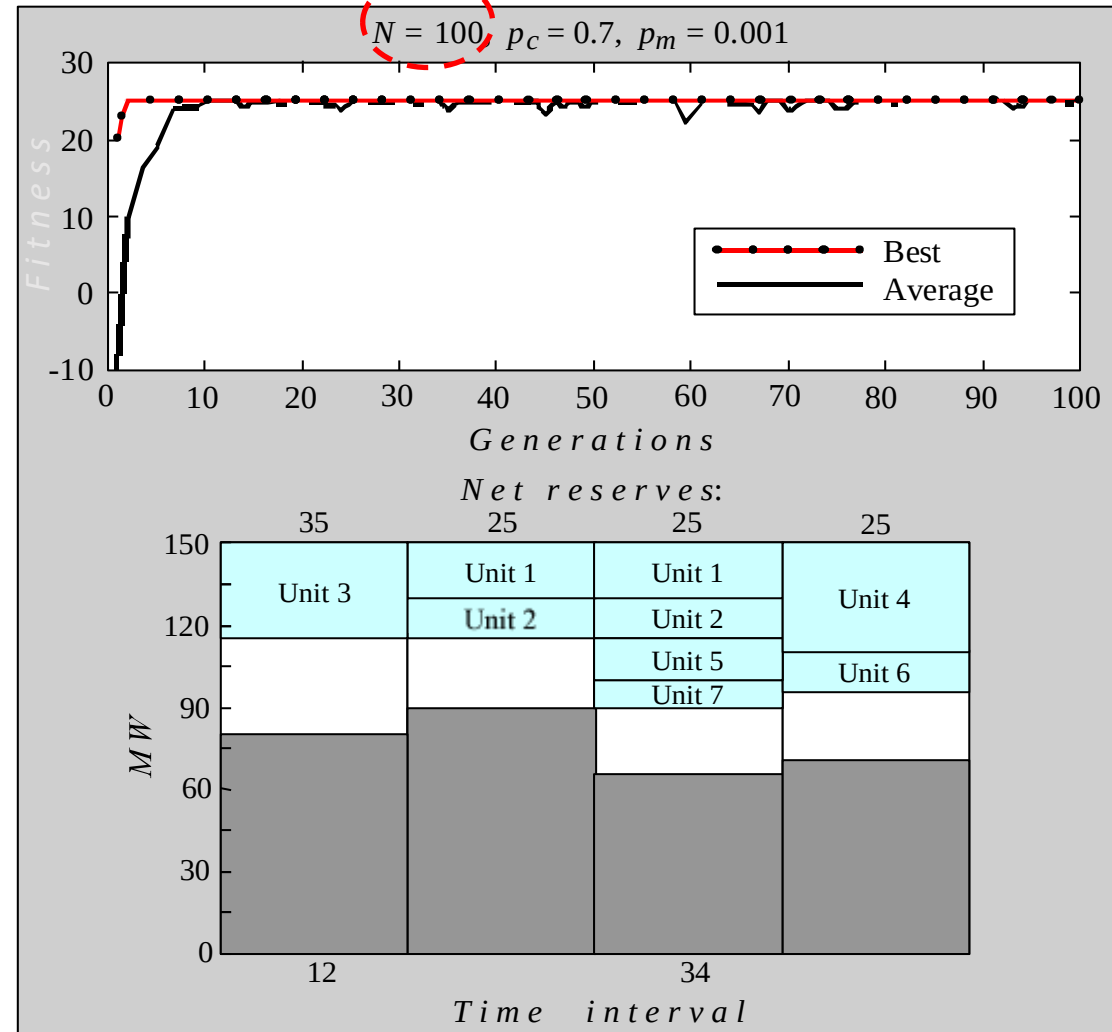
(a) 50 generations

Performance graphs and the best maintenance schedules created in a population of $N=20$ chromosomes



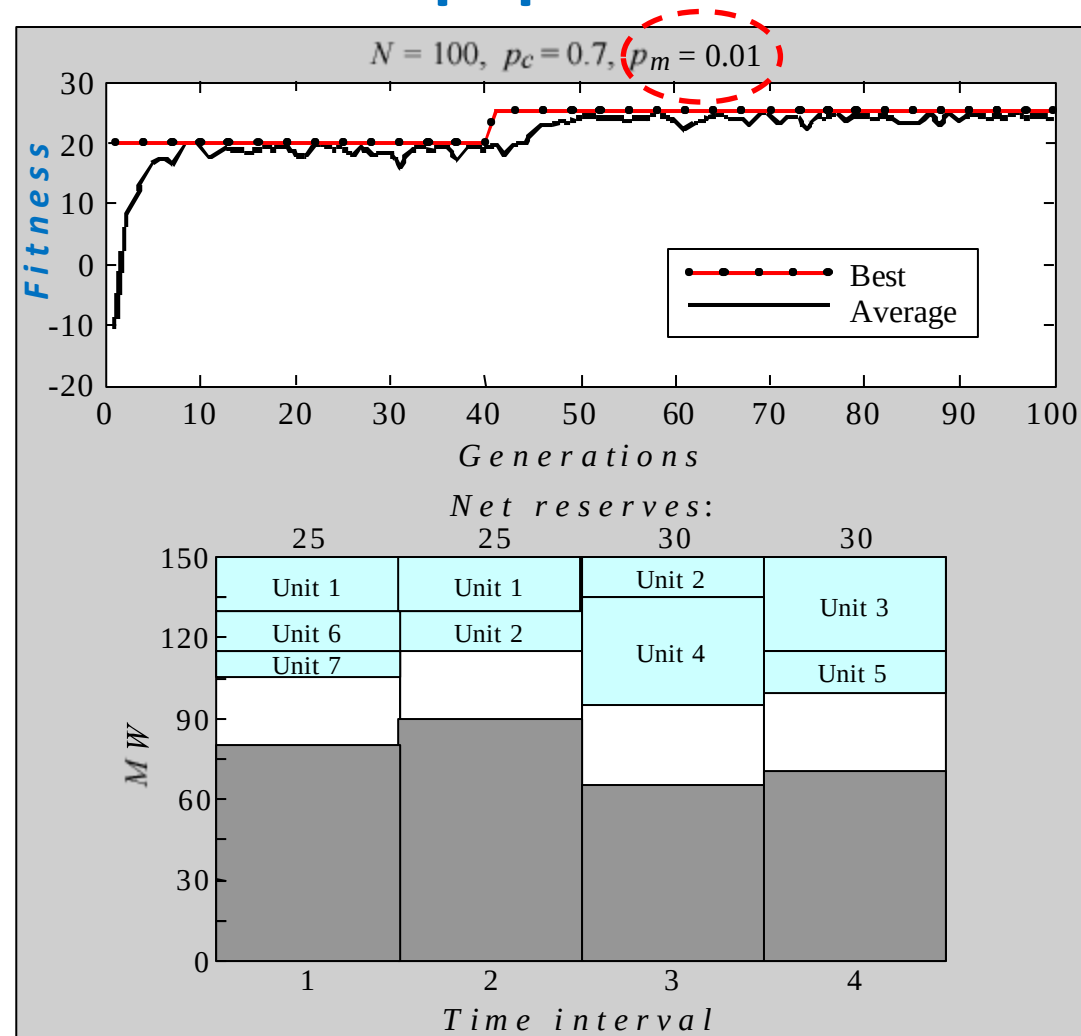
(b) 100 generations

Performance graphs and the best maintenance schedules created in a population of **N=100** chromosomes



(a) Mutation rate is 0.001

Performance graphs and the best maintenance schedules created in a population of 100 chromosomes



(b) Mutation rate is 0.01